Lab Report No 13
Understanding of Inverted Residual Block & Bottleneck

Artificial Intelligence

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

17th Jun,2025

Department of Computer Engineering,

HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)
You have to make a network in which you have 2 residual block, 2 inverted residuals and 3 bottleneck block. And you get the parameter <10 in it.
The code

```
import torch
import torch.nn as nn
import torch.nn.functional as F
class ResidualBlock(nn.Module):
    """Standard Residual Block"""
    def __init__(self, in_channels, out_channels, stride=1):
        super(ResidualBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        # Shortcut connection
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )
    def forward(self, x):
        residual = self.shortcut(x)
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += residual
        return F.relu(out)
class InvertedResidualBlock(nn.Module):
    """Inverted Residual Block (MobileNetV2 style)"""
    def __init__(self, in_channels, out_channels, stride=1, expand_ratio=6):
        super(InvertedResidualBlock, self).__init__()
        self.stride = stride
        self.use_residual = (stride == 1 and in_channels == out_channels)
        hidden_dim = in_channels * expand_ratio
        layers = []
```

```
        # Expand (pointwise)
        if expand_ratio != 1:
            layers.extend([
                nn.Conv2d(in_channels, hidden_dim, 1, bias=False),
                nn.BatchNorm2d(hidden_dim),
                nn.ReLU6(inplace=True)
            ])
        # Depthwise
        layers.extend([
            nn.Conv2d(hidden_dim, hidden_dim, 3, stride, 1, groups=hidden_dim,
bias=False),
            nn.BatchNorm2d(hidden_dim),
            nn.ReLU6(inplace=True),
            # Pointwise linear
            nn.Conv2d(hidden_dim, out_channels, 1, bias=False),
            nn.BatchNorm2d(out_channels)
        ])
        self.conv = nn.Sequential(*layers)
    def forward(self, x):
        if self.use_residual:
            return x + self.conv(x)
        else:
            return self.conv(x)
class BottleneckBlock(nn.Module):
    """Bottleneck Block (ResNet style)"""
    def __init__(self, in_channels, out_channels, stride=1, reduction=4):
        super(BottleneckBlock, self).__init__()
        mid_channels = out_channels // reduction
        self.conv1 = nn.Conv2d(in_channels, mid_channels, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(mid_channels)
        self.conv2 = nn.Conv2d(mid_channels, mid_channels, 3, stride, 1,
bias=False)
        self.bn2 = nn.BatchNorm2d(mid_channels)
        self.conv3 = nn.Conv2d(mid_channels, out_channels, 1, bias=False)
        self.bn3 = nn.BatchNorm2d(out_channels)
        # Shortcut connection
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride, bias=False),
```

```
            nn.BatchNorm2d(out_channels)
        )
    def forward(self, x):
        residual = self.shortcut(x)
        out = F.relu(self.bn1(self.conv1(x)))
        out = F.relu(self.bn2(self.conv2(out)))
        out = self.bn3(self.conv3(out))
        out += residual
        return F.relu(out)
class EfficientMixedNetwork(nn.Module):
    """Network with 2 Residual, 2 Inverted Residual, and 3 Bottleneck blocks"""
    def __init__(self, num_classes=1000):
        super(EfficientMixedNetwork, self).__init__()
        # Initial convolution
        self.conv1 = nn.Conv2d(3, 32, 7, 2, 3, bias=False)
        self.bn1 = nn.BatchNorm2d(32)
        self.maxpool = nn.MaxPool2d(3, 2, 1)
        # 2 Residual Blocks
        self.res_block1 = ResidualBlock(32, 64, stride=1)
        self.res_block2 = ResidualBlock(64, 64, stride=1)
        # 2 Inverted Residual Blocks
        self.inv_res_block1 = InvertedResidualBlock(64, 96, stride=2,
expand_ratio=3)
        self.inv_res_block2 = InvertedResidualBlock(96, 96, stride=1,
expand_ratio=3)
        # 3 Bottleneck Blocks
        self.bottleneck1 = BottleneckBlock(96, 128, stride=2, reduction=2)
        self.bottleneck2 = BottleneckBlock(128, 128, stride=1, reduction=2)
        self.bottleneck3 = BottleneckBlock(128, 256, stride=2, reduction=2)
        # Global average pooling and classifier
        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(256, num_classes)
        # Initialize weights
        self._initialize_weights()
    def _initialize_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out',
nonlinearity='relu')
            elif isinstance(m, nn.BatchNorm2d):
```

```
                nn.init.ones_(m.weight)
                nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                nn.init.zeros_(m.bias)
    def forward(self, x):
        # Initial layers
        x = F.relu(self.bn1(self.conv1(x)))
        x = self.maxpool(x)
        # Residual blocks
        x = self.res_block1(x)
        x = self.res_block2(x)
        # Inverted residual blocks
        x = self.inv_res_block1(x)
        x = self.inv_res_block2(x)
        # Bottleneck blocks
        x = self.bottleneck1(x)
        x = self.bottleneck2(x)
        x = self.bottleneck3(x)
        # Global average pooling and classification
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.fc(x)
        return x
def count_parameters(model):
    """Count the number of trainable parameters in the model"""
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
# Create the model and check parameters
def main():
    # Create model for ImageNet (1000 classes)
    model = EfficientMixedNetwork(num_classes=1000)
    # Count parameters
    total_params = count_parameters(model)
    print(f"Total trainable parameters: {total_params:,}")
    print(f"Parameters in millions: {total_params/1e6:.2f}M")
    # Test with a sample input
    x = torch.randn(1, 3, 224, 224)  # Batch size 1, 3 channels, 224x224 image
    with torch.no_grad():
        output = model(x)
        print(f"Output shape: {output.shape}")
```

```
    # Model summary
    print("\n=== Model Architecture ===")
    print("1. Initial Conv + MaxPool")
    print("2. Residual Block 1 (32→64)")
    print("3. Residual Block 2 (64→64)")
    print("4. Inverted Residual Block 1 (64→96, stride=2)")
    print("5. Inverted Residual Block 2 (96→96)")
    print("6. Bottleneck Block 1 (96→128, stride=2)")
    print("7. Bottleneck Block 2 (128→128)")
    print("8. Bottleneck Block 3 (128→256, stride=2)")
    print("9. Global Average Pool + FC")
    return model
if __name__ == "__main__":
    model = main()
```

The results (Screenshot)

```
Total trainable parameters: 835,048
Parameters in millions: 0.84M
Output shape: torch.Size([1, 1000])

=== Model Architecture ===
1. Initial Conv + MaxPool
2. Residual Block 1 (32→64)
3. Residual Block 2 (64→64)
4. Inverted Residual Block 1 (64→96, stride=2)
5. Inverted Residual Block 2 (96→96)
6. Bottleneck Block 1 (96→128, stride=2)
7. Bottleneck Block 2 (128→128)
8. Bottleneck Block 3 (128→256, stride=2)
9. Global Average Pool + FC
```

Conclusion

In conclusion, the Inverted Residual Block increases channels, depth wise convolves, and projects them back into fewer channels, which makes it ideal for mobile networks, like MobileNetV2.Bottleneck Block compresses and convolves features and expands them again we used by deep networks such as ResNet to cut down on computation but still keep the same performance. Onto representational capacity with fewer parameters and costs at a less computation level. They create a wider and faster network through the residual learning and flow of useful information in it.